

Przeszkody dla Lamy

Lama Beza chce podróżować po Wyżynie Andyjskiej. Ma mapę wyżyny w postaci siatki złożonej z $N \times M$ kwadratowych komórek. Wiersze mapy są ponumerowane od 0 do $N - 1$ od góry do dołu, a kolumny od 0 do $M - 1$ od lewej do prawej. Komórka mapy w wierszu i oraz kolumnie j ($0 \leq i < N, 0 \leq j < M$) jest oznaczona jako (i, j) .

Lama zbadała klimat wyżyny i odkryła, że wszystkie komórki w każdym wierszu mapy mają taką samą **temperaturę**, a wszystkie komórki w każdej kolumnie mapy mają taką samą **wilgotność**. Lama podała Ci dwie tablice liczb całkowitych T i H o długościach odpowiednio N i M , gdzie $T[i]$ ($0 \leq i < N$) oznacza temperaturę komórek w wierszu i , a $H[j]$ ($0 \leq j < M$) oznacza wilgotność komórek w kolumnie j .

Lama zbadała również florę płaskowyżu i zauważyła, że komórka (i, j) jest **wolna od roślinności** wtedy i tylko wtedy, gdy jej temperatura jest większa niż jej wilgotność, formalnie $T[i] > H[j]$.

Lama może podróżować po płaskowyżu tylko podążając **prawidłowymi ścieżkami**. Prawidłowa ścieżka to sekwencja parami różnych komórek, które spełniają następujące warunki:

- Każda para kolejnych komórek na ścieżce ma wspólny bok.
- Wszystkie komórki na ścieżce są wolne od roślinności.

Twoim zadaniem jest udzielenie odpowiedzi na Q zapytań. Każde zapytanie jest opisane przez cztery liczby całkowite: L, R, S oraz D . Oznaczają one, że musisz określić, czy istnieje prawidłowa ścieżka, która spełnia poniższe warunki:

- Ścieżka zaczyna się w komórce $(0, S)$ i kończy w komórce $(0, D)$.
- Wszystkie komórki na ścieżce leżą w kolumnach od L do R włącznie.

Możesz założyć, że zarówno $(0, S)$ jak i $(0, D)$ są wolne od roślinności.

Szczegóły implementacji

Pierwszą procedurą, którą powinieneś zaimplementować jest:

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : tablica o długości N opisująca temperaturę w każdym wierszu.
- H : tablica o długości M opisująca wilgotność w każdej kolumnie.

- Procedura ta jest wywoływana dokładnie raz dla każdego przypadku testowego, przed jakimkolwiek wywołaniem `can_reach`.

Drugą procedurą, którą powinieneś zaimplementować jest:

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : liczby całkowite opisujące zapytanie.
- Ta procedura jest wywoływana Q razy dla każdego przypadku testowego.

Procedura powinna zwrócić `true` wtedy i tylko wtedy, gdy istnieje prawidłowa ścieżka z komórki $(0, S)$ do komórki $(0, D)$, taka, że wszystkie komórki na ścieżce leżą w kolumnach od L do R włącznie.

Ograniczenia

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ dla każdego i takiego, że $0 \leq i < N$.
- $0 \leq H[j] \leq 10^9$ dla każdego j takiego, że $0 \leq j < M$.
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- Obie komórki $(0, S)$ i $(0, D)$ są wolne od roślinności.

Podzadania

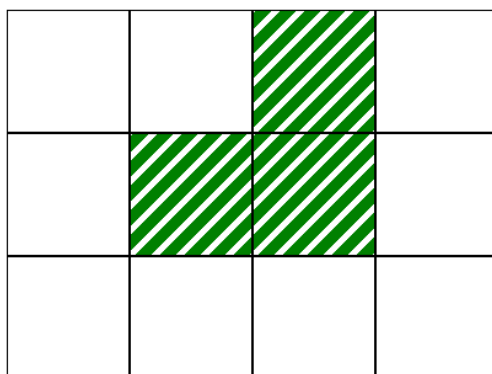
Podzadanie	Punktacja	Dodatkowe ograniczenia
1	10	$L = 0, R = M - 1$ dla każdego zapytania. $N = 1$
2	14	$L = 0, R = M - 1$ dla każdego zapytania. $T[i - 1] \leq T[i]$ dla każdego i takiego, że $1 \leq i < N$.
3	13	$L = 0, R = M - 1$ dla każdego zapytania. $T = [2, 1, 3]$.
4	21	$L = 0, R = M - 1$ dla każdego zapytania. $Q \leq 10$.
5	25	$L = 0, R = M - 1$ dla każdego zapytania.
6	17	Brak dodatkowych ograniczeń.

Przykład

Rozważmy następujące wywołanie:

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

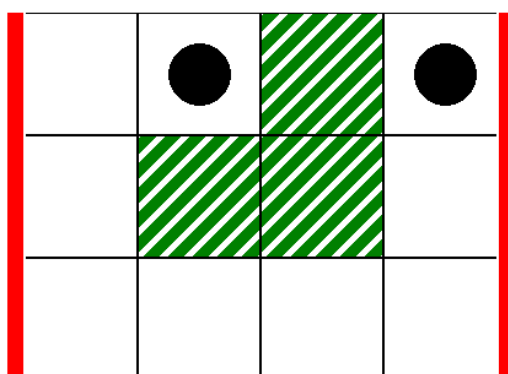
Odpowiada to mapie przedstawionej na poniższym obrazku, gdzie białe komórki są wolne od roślinności:



Jako pierwsze zapytanie rozważ następujące wywołanie:

```
can_reach(0, 3, 1, 3)
```

Odpowiada to scenariuszowi na poniższym obrazku, gdzie grube pionowe linie wskazują zakres kolumn od $L = 0$ do $R = 3$, a czarne kółka wskazują początkowe i końcowe komórki:



W tym przypadku lama może dotrzeć z komórki (0,1) do komórki (0,3) następującą prawidłową ścieżką:

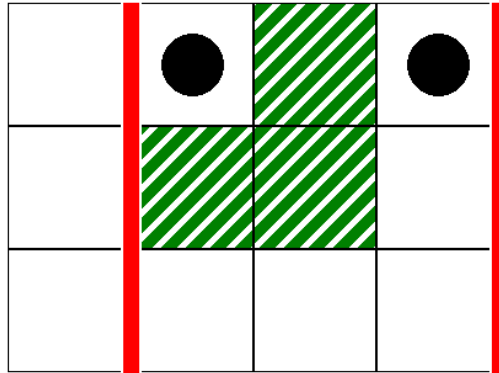
$(0, 1), (0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (1, 3), (0, 3)$

Dlatego to wywołanie powinno zwrócić `true`.

Jako drugie zapytanie rozważ następujące wywołanie:

```
can_reach(1, 3, 1, 3)
```

Odpowiada to scenariuszowi przedstawionemu na poniższym obrazku:



W tym przypadku nie istnieje prawidłowa ścieżka z komórki (0,1) do komórki (0,3) taka, że komórki na ścieżce znajdują się w kolumnach od 1 do 3 włącznie. Dlatego to wywołanie powinno zwrócić `false`.

Przykładowy program oceniający

Format wejściowy:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

Gdzie $L[k], R[k], S[k]$ i $D[k]$ ($0 \leq k < Q$) określają parametry dla każdego wywołania `can_reach`.

Format wyjściowy:

```
A[0]
A[1]
...
A[Q-1]
```

Gdzie $A[k]$ ($0 \leq k < Q$) jest równe 1 jeśli wywołanie `can_reach(L[k], R[k], S[k], D[k])` zwróciło `true`, a 0 w przeciwnym przypadku.